React.js Developer Interview Preparation Guide

React | JavaScript | JSX | Hooks | State Management | Rendering & Performance

Arrays & Lists | Forms & Events | React Router

This guide provides concise, question-and-answer format responses for React.js interview preparation. Each answer includes a clear explanation and a minimal working code example to illustrate the concept in practice.

Topics Covered

1. Basic React
2. React Hooks
3. State Management
4. Rendering & Performance
5. Arrays & Lists
6. Forms & Events
7. React Router

How to use this guide

- Answers are written for 1-2 minute spoken replies in an interview setting.
- Code snippets show minimal working patterns — focus on the concept, not boilerplate.
- Each section builds on the previous one; review topics in order for best results.
- Use the code examples as reference implementations you can adapt to your projects.
- Practice explaining each answer out loud to build confidence and fluency.

Basic React

Q1. How does React.js work?

React is a declarative, component-based JavaScript library for building user interfaces. Instead of directly manipulating the browser DOM, React maintains an in-memory representation called the Virtual DOM. When the state of a component changes, React creates a new Virtual DOM tree and compares it with the previous one using a process called reconciliation (diffing). It then computes the minimal set of changes and applies only those to the real DOM. This batching and selective updating makes UI updates fast and predictable.

React components are written as functions (or classes) that return JSX. Data flows down from parent to child via props, and state is managed within components. React re-renders only the components whose state or props changed, keeping the view always in sync with the underlying data.

Example: A component whose state change triggers a re-render

```
// A simple functional component
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Q2. What is JSX and how is it converted into JavaScript?

JSX is a syntax extension for JavaScript that lets you write HTML-like markup inside JavaScript. Under the hood, JSX is not valid JavaScript; tools like Babel transpile it into React.createElement() calls, which return plain JavaScript objects called React elements.

Each JSX tag becomes a function call. Attributes become props, and children become nested arguments. This is why you must import React (in older setups) and why JSX expressions are just objects describing what the UI should look like.

JSX -> createElement -> element object

```
// JSX you write
const element = <h1 className="title">Hello, React!</h1>;

// Gets compiled to:
const element = React.createElement(
  'h1',
  { className: 'title' },
  'Hello, React!'
);

// Which produces a React element object:
// { type: 'h1', props: { className: 'title', children: 'Hello, React!' } }
```

Q3. What is a React component?

A React component is a reusable, independent piece of UI defined as a JavaScript function (or class) that returns JSX. Components let you split the UI into small, composable units. They accept inputs called props and manage their own internal state.

Components are the building blocks of a React app. You compose them together to form complex UIs, and React handles rendering and updating them when data changes.

A component that receives props and renders markup

```
// Functional component
function Welcome({ name }) {
  return <h1>Hello, {name}!</h1>;
}

// Usage
<Welcome name="Aarav" />
```

Q4. Difference between functional and class components in React?

Functional components are plain JavaScript functions that return JSX. Before Hooks (React 16.8), they could not hold state or lifecycle logic; class components were required for that. With Hooks, function components can now use state (useState), side effects (useEffect), context, refs, and more, making class components largely unnecessary in modern React.

Class components extend React.Component, define a render() method, use this.state and this.setState, and have lifecycle methods like componentDidMount. Function components are simpler, have less boilerplate, are easier to test, and encourage better code reuse through custom hooks.

Class vs functional component doing the same thing

```
// Class component
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }
  componentDidMount() { console.log('mounted'); }
  render() {
    return (
      <button onClick={() => this.setState({ count: this.state.count + 1 })}>
        {this.state.count}
      </button>
    );
  }
}

// Equivalent functional component (modern)
function Counter() {
  const [count, setCount] = useState(0);
  useEffect(() => { console.log('mounted'); }, []);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

Q5. What are props and default props in React?

Props (properties) are read-only inputs passed from a parent component to a child. They allow data to flow down the component tree. Default props provide fallback values when a prop is not explicitly passed.

In modern React with function components, default values are set using default parameter values or destructuring defaults. The older static defaultProps syntax still works for class components but is being deprecated for function components.

Setting default props in function and class components

```
// Default props via destructuring defaults
function Greeting({ name = 'Guest', lang = 'en' }) {
  return <p>Hello, {name}! ({lang})</p>;
}

// Usage
<Greeting /> // Hello, Guest! (en)
```

```
<Greeting name="Priya" /> // Hello, Priya! (en)

// Older class-based approach
class Greeting extends React.Component {
  static defaultProps = { name: 'Guest' };
  render() { return <p>Hello, {this.props.name}</p>; }
}
```

Q6. What is state in React and how do you update it?

State is internal data owned by a component that, when changed, triggers a re-render. Unlike props (passed in), state is managed within the component. In function components, state is created with the useState hook. State updates are asynchronous and batched; you should never mutate state directly but always use the setter function.

When the new state depends on the previous state, pass a callback to the setter to avoid stale closures. React merges or replaces based on how you structure the update.

Using useState with a functional update

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  // Correct: functional update when depending on previous value
  const increment = () => setCount(prev => prev + 1);

  // Wrong: direct mutation (never do this)
  // count++; // does NOT trigger re-render

  return <button onClick={increment}>{count}</button>;
}
```

Q7. Difference between props and state in React?

Props are passed from parent to child and are read-only inside the receiving component. State is local, owned, and managed by the component itself; it can change over time and triggers re-renders. Props make components reusable by externalising configuration, while state makes components interactive by holding data that changes.

A child cannot change its props directly; to change them it must ask the parent (via a callback prop). State, however, is changed internally by the component that owns it.

Parent holds state, child receives props

```
function Child({ label, onClick }) {
  // props: 'label' is read-only, 'onClick' notifies parent
  return <button onClick={onClick}>{label}</button>;
}

function Parent() {
  const [count, setCount] = useState(0); // state: owned by Parent
  return (
    <Child
      label={`Count: ${count}`}
      onClick={() => setCount(count + 1)}
    />
  );
}
```

Q8. What are fragments in React?

Fragments let you group multiple elements without adding an extra DOM node. Returning sibling JSX normally requires a wrapping element, which pollutes the DOM with unnecessary divs. Fragments solve this cleanly. You can use the explicit <React.Fragment> syntax (which accepts a key prop) or the shorthand <>...</> (which does not accept props).

Three ways to use fragments

```
import React, { Fragment } from 'react';

function List() {
  return (
    <React.Fragment>
      <h1>Title</h1>
      <p>Description</p>
    </React.Fragment>
  );
}

// Shorthand (no key allowed)
function Row() {
  return (
    <>
      <td>Cell 1</td>
      <td>Cell 2</td>
    </>
  );
}

// Fragment with key (needed in lists)
function Items({ data }) {
  return data.map(item => (
    <Fragment key={item.id}>
      <dt>{item.term}</dt>
      <dd>{item.desc}</dd>
    </Fragment>
  ));
}
```

Q9. What is the difference between controlled and uncontrolled components?

A controlled component has its form data driven by React state; the input's value is set from state and every change updates state. React is the single source of truth. An uncontrolled component stores its own data in the DOM (via a ref), and React reads it when needed (e.g. on submit). Controlled components give you instant validation and control; uncontrolled components are simpler for one-time reads and integrate easily with non-React code.

Controlled (state-driven) vs uncontrolled (ref-driven)

```
import { useState, useRef } from 'react';

// Controlled: value bound to state
function ControlledForm() {
  const [name, setName] = useState('');
  return (
    <input
      value={name}
      onChange={e => setName(e.target.value)}
    />
  );
}

// Uncontrolled: value read from DOM via ref
function UncontrolledForm() {
  const inputRef = useRef(null);
  const handleSubmit = (e) => {
```

```
    e.preventDefault();
    console.log(inputRef.current.value);
  };
  return (
    <form onSubmit={handleSubmit}>
      <input ref={inputRef} defaultValue="default" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

Q10. How does the DOM manage the input value in uncontrolled components?

In an uncontrolled component, the browser DOM itself holds the current value of the input. React does not track every keystroke in state. Instead, you access the value through a ref pointing to the DOM node. The input behaves like a standard HTML element, and its value persists in the DOM between renders. You can set an initial value with the defaultValue prop; after that the user's typing lives only in the DOM until you read it.

Reading an uncontrolled input value via ref

```
import { useRef } from 'react';

function Profile() {
  const emailRef = useRef(null);

  const save = () => {
    // Read the current value directly from the DOM node
    console.log('Email:', emailRef.current.value);
  };

  return (
    <>
      <input
        ref={emailRef}
        type="email"
        defaultValue="user@example.com"
      />
      <button onClick={save}>Save</button>
    </>
  );
}
```

React Hooks

Q11. What are Hooks in React and why were they introduced?

Hooks are special functions (names starting with 'use') that let function components use state and lifecycle features previously available only in class components. They were introduced in React 16.8 to solve several problems: (1) reusing stateful logic was hard with class components (HOCs and render props created 'wrapper hell'), (2) complex lifecycle methods split related logic apart, and (3) classes were confusing for humans and tools. Hooks let you extract reusable logic into custom hooks and keep related code together.

A custom hook reusing stateful logic

```
import { useState, useEffect } from 'react';

function useWindowWidth() { // custom hook
  const [width, setWidth] = useState(window.innerWidth);
  useEffect(() => {
    const onResize = () => setWidth(window.innerWidth);
```

```
    window.addEventListener('resize', onResize);
    return () => window.removeEventListener('resize', onResize);
  }, []);
  return width;
}

function Responsive() {
  const width = useWindowWidth(); // reuse logic cleanly
  return <p>Window is {width}px wide</p>;
}
```

Q12. How does the useState hook work?

useState returns a pair: the current state value and a function to update it. On the first render the state is initialised to the argument you pass (or the result of calling an initialiser function for lazy initialisation). When you call the setter, React schedules a re-render with the new value. If you pass a function to the setter, it receives the previous state, which avoids stale-value bugs in rapid successive updates.

useState with lazy init and functional updates

```
import { useState } from 'react';

function Timer() {
  // lazy initialisation: function runs once
  const [seconds, setSeconds] = useState(
    () => Number(localStorage.getItem('sec')) || 0
  );

  const tick = () => setSeconds(prev => prev + 1); // functional update
  const reset = () => setSeconds(0); // direct value

  return (
    <>
      <p>{seconds}s</p>
      <button onClick={tick}>+1</button>
      <button onClick={reset}>Reset</button>
    </>
  );
}
```

Q13. What is useEffect in React and what is the role of its dependency array?

useEffect lets you perform side effects (data fetching, subscriptions, DOM manipulation, timers) in function components. It runs after render. The dependency array (second argument) controls when the effect re-runs: no array means it runs after every render; an empty array [] runs only once after mount; an array with values runs when any of those values change. The function you return from useEffect is a cleanup that runs before the next effect and on unmount.

useEffect with dependency array and cleanup

```
import { useState, useEffect } from 'react';

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    let cancelled = false;
    fetch(`/api/users/${userId}`)
      .then(r => r.json())
      .then(data => { if (!cancelled) setUser(data); });
    // cleanup: avoid setting state after unmount
    return () => { cancelled = true; };
  }, [userId]); // re-run only when userId changes
```

```
  if (!user) return <p>Loading...</p>;
  return <p>{user.name}</p>;
}
```

Q14. What is the difference between useEffect and useLayoutEffect?

Both have the same signature, but they fire at different points. useEffect runs asynchronously after the browser paints, so it does not block visual updates, ideal for most side effects. useLayoutEffect runs synchronously after DOM mutations but before the browser paints, so it is used when you need to read or change layout (e.g. measuring an element, preventing flicker) before the user sees it. useLayoutEffect can hurt performance, so prefer useEffect unless you see a visual flicker.

useLayoutEffect for measuring DOM before paint

```
import { useRef, useState, useLayoutEffect } from 'react';

function Tooltip({ children }) {
  const ref = useRef(null);
  const [pos, setPos] = useState({ top: 0, left: 0 });

  // useLayoutEffect: measure before paint to avoid flicker
  useLayoutEffect(() => {
    const rect = ref.current.getBoundingClientRect();
    setPos({ top: rect.bottom, left: rect.left });
  }, [children]);

  return (
    <>
      <span ref={ref}>{children}</span>
      <div style={{ position: 'absolute', top: pos.top, left: pos.left }}>
        Tooltip content
      </div>
    </>
  );
}
```

Q15. What is the useContext hook and when should you use it?

useContext reads and subscribes to a React Context, letting a component access a value deep in the tree without manually passing props through every level ('prop drilling'). You create context with createContext, provide a value high in the tree with a Provider, and consume it anywhere below with useContext. Use it for data that is global to a subtree (theme, locale, authenticated user, app config) but not for high-frequency updates, because every consumer re-renders when the value changes.

Create, provide, and consume context

```
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext('light');

function App() {
  const [theme, setTheme] = useState('dark');
  return (
    <ThemeContext.Provider value={theme}>
      <Toolbar />
      <button onClick={() =>
        setTheme(t => t === 'dark' ? 'light' : 'dark')
      }>Toggle</button>
    </ThemeContext.Provider>
  );
}
```

```
function Toolbar() {
  return <ThemedButton />; // no props drilled
}

function ThemedButton() {
  const theme = useContext(ThemeContext);
  return <button className={theme}>Click me</button>;
}
```

Q16. What is the useReducer hook and when is it preferred over useState?

useReducer is an alternative to useState for managing complex state logic. It takes a reducer function (state, action) => newState and an initial state, and returns the current state and a dispatch function. Prefer useReducer when: state transitions follow clear rules, the next state depends on multiple related values, you need to update state deep in a tree, or you want testable, predictable updates. It also lets you pass dispatch down instead of multiple callbacks.

useReducer with a todos reducer

```
import { useReducer } from 'react';

function todosReducer(state, action) {
  switch (action.type) {
    case 'add':
      return [...state, { id: Date.now(), text: action.text, done: false }];
    case 'toggle':
      return state.map(t =>
        t.id === action.id ? { ...t, done: !t.done } : t
      );
    case 'remove':
      return state.filter(t => t.id !== action.id);
    default:
      return state;
  }
}

function TodoApp() {
  const [todos, dispatch] = useReducer(todosReducer, []);
  return (
    <>
      <button onClick={() =>
        dispatch({ type: 'add', text: 'Learn React' })
      }>Add</button>
      <ul>
        {todos.map(t => (
          <li key={t.id} onClick={() =>
            dispatch({ type: 'toggle', id: t.id })
          }>
            {t.text} {t.done ? 'DONE' : ''}
          </li>
        ))}
      </ul>
    </>
  );
}
```

Q17. What is the useRef hook and what are its common use cases?

useRef returns a mutable object whose .current property persists across renders without causing re-renders when changed. Its two main uses are: (1) accessing and manipulating DOM nodes directly (e.g. focusing an input, measuring an element), and (2) storing any mutable value that should survive re-renders but should not trigger them (e.g. a timer id, a previous value, an 'is mounted' flag). Unlike state, updating ref.current does not re-render.

useRef for DOM access and a timer id

```
import { useRef, useState, useEffect } from 'react';

function FocusInput() {
  const inputRef = useRef(null); // DOM reference
  const timerRef = useRef(null); // mutable non-render value

  useEffect(() => {
    inputRef.current.focus(); // focus on mount
    timerRef.current = setInterval(
      () => console.log('tick'), 1000
    );
    return () => clearInterval(timerRef.current);
  }, []);

  return <input ref={inputRef} />;
}
```

Q18. Explain the difference between useMemo and useCallback.

Both memoize values across re-renders to avoid recomputing. useMemo caches the result of a computation, returning the same memoised value until its dependencies change. useCallback caches a function reference, returning the same function instance until dependencies change. Use useMemo for expensive calculations; use useCallback to keep a stable function identity so child components that depend on it don't needlessly re-render. They are both optimisations, not semantic necessities, and should be used judiciously.

useMemo caches a value, useCallback caches a function

```
import { useMemo, useCallback, useState } from 'react';

function ProductList({ products }) {
  const [query, setQuery] = useState('');

  // useMemo: cache expensive filtered result
  const filtered = useMemo(
    () => products.filter(p =>
      p.name.toLowerCase().includes(query.toLowerCase())
    ),
    [products, query]
  );

  // useCallback: stable handler for child memoised component
  const handleClick = useCallback((id) => {
    console.log('Selected', id);
  }, []);

  return filtered.map(p => (
    <MemoItem key={p.id} product={p} onClick={handleClick} />
  ));
}
```

Q19. What are custom hooks in React and how do you create one?

A custom hook is a JavaScript function whose name starts with 'use' that encapsulates reusable stateful logic using other hooks. It lets you share logic between components without changing your component hierarchy (no wrappers needed). Custom hooks can call useState, useEffect, useContext, or any other hook. They follow the same rules of hooks. The convention is that inputs are arguments and outputs are returned values.

A useFetch custom hook reused across components

```
import { useState, useEffect } from 'react';

// Custom hook: reusable data-fetching logic
function useFetch(url) {
  const [data, setData] = useState(null);
```

```
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let active = true;
    setLoading(true);
    fetch(url)
      .then(res => res.json())
      .then(json => {
        if (active) { setData(json); setError(null); }
      })
      .catch(err => { if (active) setError(err.message); })
      .finally(() => { if (active) setLoading(false); });
    return () => { active = false; };
  }, [url]);

  return { data, loading, error };
}

// Usage in any component
function Users() {
  const { data, loading, error } = useFetch('/api/users');
  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return <ul>{data.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

Q20. What are the rules of hooks and why are they important?

React enforces two rules. First, only call hooks at the top level of your component or custom hook, never inside loops, conditions, or nested functions; React relies on call order to associate each hook with its state, so conditional calls break that mapping. Second, only call hooks from React function components or other custom hooks, never from regular JavaScript functions. These rules exist because hooks are stored in a fixed-order list per component; breaking them causes state to get mismatched and bugs that are hard to trace.

Wrong vs correct hook usage

```
import { useState, useEffect } from 'react';

function Bad({ user }) {
  // WRONG: hook inside a condition
  if (user) {
    const [name, setName] = useState(''); // order changes -> bug
  }
}

function Good({ user }) {
  const [name, setName] = useState(''); // always called first
  useEffect(() => {
    if (user) setName(user.name); // condition inside
  }, [user]);
}
```

State Management

Q21. What is State Management in React and what is the difference between Local and Global State?

State management is the way an application stores, updates, and shares data across its components. Local state lives inside a single component (e.g. useState) and is not visible elsewhere. Global state is shared across many components, often far apart in the tree, and is usually placed in a store or context so that any component can read or update it. Use local state for UI concerns of one component; use global state for data many components need, such as the logged-in user, cart, or theme.

Local state vs global state via context

```
// Local state: only Counter needs it
function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
}

// Global state via Context: shared across the tree
const CartContext = createContext();
function App() {
  const [cart, setCart] = useState([]);
  return (
    <CartContext.Provider value={{ cart, setCart }}>
      <Header />
      <ProductGrid />
      <Checkout />
    </CartContext.Provider>
  );
}
```

Q22. What are some popular state management solutions in React/Next.js?

Several libraries serve different needs. Redux Toolkit is the standard for large apps with complex, predictable state flows; Zustand offers a minimal, hook-based API with less boilerplate; Jotai provides atomic state management; Recoil (now less active) pioneered atoms; React Context + useReducer is the built-in option for moderate needs. For server state, React Query (TanStack Query) and SWR handle caching, fetching, and synchronisation. Redux Toolkit + RTK Query is a common full-stack pairing in Next.js.

Zustand for client state, React Query for server state

```
// Zustand: minimal global store
import { create } from 'zustand';

const useStore = create((set) => ({
  count: 0,
  increment: () => set(state => ({ count: state.count + 1 })),
}));

function Counter() {
  const { count, increment } = useStore();
  return <button onClick={increment}>{count}</button>;
}

// React Query: server state
import { useQuery } from '@tanstack/react-query';
function Users() {
  const { data } = useQuery(
    ['users'],
    () => fetch('/api/users').then(r => r.json())
```

```
  );
  return <ul>{data?.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

Q23. When should you use Redux over Context API?

Use Redux when your app has complex state with many interacting updates, needs time-travel debugging, requires middleware (thunks, sagas) for async logic, or has high-frequency updates that would cause Context to re-render too many consumers. Context is fine for low-frequency, app-wide values (theme, auth, locale) where simplicity matters. Redux Toolkit adds structure, devtools, and performance optimisations that scale better for large applications.

Context for simple, Redux Toolkit for complex

```
// Context: fine for low-frequency global values
const ThemeContext = createContext();
<ThemeContext.Provider value={theme}>
  <App />
</ThemeContext.Provider>

// Redux Toolkit: better for complex, high-frequency state
import { configureStore, createSlice } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',
  initialState: [],
  reducers: {
    addItem: (state, action) => { state.push(action.payload); },
    removeItem: (state, action) =>
      state.filter(i => i.id !== action.payload.id),
  },
});

const store = configureStore({
  reducer: { cart: cartSlice.reducer }
});
```

Q24. What is the difference between Client State and Server State?

Client state is data owned and managed in the browser (form inputs, UI toggles, local filters). Server state is data that lives on a remote server; it is asynchronous, can be stale, may be shared across users, and needs caching, refetching, and synchronisation. Managing server state with local useState leads to bugs (no cache, no invalidation). Libraries like React Query and SWR are purpose-built for server state, while Redux/Zustand suit client state.

Server state handled properly with React Query

```
// Wrong: treating server data as client state
function WrongUsers() {
  const [users, setUsers] = useState([]);
  useEffect(() => {
    fetch('/api/users').then(r => r.json()).then(setUsers);
  }, []);
  // no caching, no refetch, no invalidation, stale risk
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

// Right: React Query handles server state
import { useQuery } from '@tanstack/react-query';
function RightUsers() {
  const { data } = useQuery(['users'], fetchUsers, {
    staleTime: 30000, // cache 30s
    refetchOnWindowFocus: true, // auto-refresh
  });
  return <ul>{data?.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
```

```
}
```

Q25. Explain the Context API in React.

The Context API is React's built-in way to share values across the component tree without prop drilling. You create a context with createContext(defaultValue), provide a value to a subtree using a Context.Provider, and consume it in any descendant with useContext or the Consumer component. When the provider value changes, every consumer re-renders. It is well suited for themes, auth, locale, and other low-to-medium frequency global data.

Create -> provide -> consume a context

```
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext(null); // 1. create

function App() {
  const [user, setUser] = useState(null);
  return (
    <AuthContext.Provider value={{ user, setUser }}>
      <Navbar />
      <Dashboard />
    </AuthContext.Provider>
  );
}

function Navbar() {
  const { user, setUser } = useContext(AuthContext);
  return user
    ? <button onClick={() => setUser(null)}>
        Logout {user.name}
      </button>
    : <button onClick={() => setUser({ name: 'Aarav' })}>
        Login
      </button>;
}
```

Q26. What is the role of useReducer in state management?

useReducer centralises state-update logic in a single reducer function, making complex state transitions predictable, testable, and traceable. Instead of scattered setState calls, components dispatch actions that describe what happened, and the reducer decides the next state. This pattern shines for multi-field forms, wizards, or any state where the next value depends on several factors. It also pairs naturally with Context to create a lightweight global store.

useReducer + Context as a mini store

```
import { useReducer, createContext, useContext } from 'react';

function cartReducer(state, action) {
  switch (action.type) {
    case 'add': return [...state, action.item];
    case 'clear': return [];
    case 'remove': return state.filter(i => i.id !== action.id);
    default: return state;
  }
}

const CartContext = createContext();
function CartProvider({ children }) {
  const [cart, dispatch] = useReducer(cartReducer, []);
  return (
    <CartContext.Provider value={{ cart, dispatch }}>
      {children}
    </CartContext.Provider>
```

```
  );
}

function AddButton({ item }) {
  const { dispatch } = useContext(CartContext);
  return (
    <button onClick={() => dispatch({ type: 'add', item })}>
      Add to cart
    </button>
  );
}
```

Q27. How do you handle persistent state in React apps?

Persistent state survives page reloads by being stored outside React, usually in localStorage (synchronous, ~5MB) or sessionStorage for the tab's lifetime. You initialise state from storage, and write back whenever it changes. For larger or cross-device data, use IndexedDB or a backend. A common pattern is a custom hook that syncs a useState to localStorage, and for server data, libraries like Redux-Persist or React Query's persisters.

A usePersistentState hook backed by localStorage

```
import { useState, useEffect } from 'react';

// Custom hook: state persisted to localStorage
function usePersistentState(key, initial) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved ? JSON.parse(saved) : initial;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

function Settings() {
  const [theme, setTheme] = usePersistentState('theme', 'light');
  return (
    <button onClick={() =>
      setTheme(t => t === 'light' ? 'dark' : 'light')
    }>
      Theme: {theme}
    </button>
  );
}
```

Q28. What is the difference between derived state and computed state in React?

These terms are often used interchangeably, both describing values calculated from existing state or props rather than stored separately. Derived/computed state is recalculated on each render from other state; it should not be stored in its own useState because that creates a synchronisation burden and can go stale. Instead, compute it inline or wrap it in useMemo when expensive. The key principle: don't duplicate data in state if it can be derived.

Deriving values instead of storing them

```
import { useState, useMemo } from 'react';

function Cart({ items }) {
  // Derived inline: recomputed each render from items
  const totalItems = items.reduce((sum, i) => sum + i.qty, 0);
```

```
  // Expensive derivation memoised
  const totalPrice = useMemo(
    () => items.reduce((sum, i) => sum + i.qty * i.price, 0),
    [items]
  );

  // WRONG: storing derived value in state -> can go stale
  // const [total, setTotal] = useState(0);
  // useEffect(() => setTotal(items.reduce(...)), [items]);

  return <p>{totalItems} items, total Rs.{totalPrice}</p>;
}
```

Q29. How can you optimize state updates for performance in React?

Several practices help. Keep state as local as possible so updates re-render few components. Use functional updates to avoid stale values. Batch updates (React 18 auto-batches even in promises/timeouts). Memoise expensive derived values with useMemo, keep handler identities stable with useCallback, and wrap heavy child components in React.memo. For global state, prefer libraries that select slices (Redux, Zustand) so only components using changed data re-render. Move rapidly changing values to refs when they don't affect the UI.

Memoising values, handlers, and components

```
import { useState, useMemo, useCallback, memo } from 'react';

const ExpensiveChild = memo(function Child({ value, onSelect }) {
  return <li onClick={() => onSelect(value)}>{value}</li>;
});

function List({ items }) {
  const [query, setQuery] = useState('');

  const filtered = useMemo(
    () => items.filter(i => i.includes(query)),
    [items, query]
  );

  // stable identity -> memoised children don't re-render
  const handleSelect = useCallback((v) => console.log(v), []);

  return (
    <>
      <input value={query}
             onChange={e => setQuery(e.target.value)} />
      <ul>
        {filtered.map(i => (
          <ExpensiveChild key={i} value={i} onSelect={handleSelect} />
        ))}
      </ul>
    </>
  );
}
```

Q30. How do you handle asynchronous state updates in React?

Async updates (from fetches, timers, promises) must use the setter inside the async callback, not mutate state directly. Guard against setting state after unmount (use a flag or cleanup in useEffect). For complex async flows, useReducer with async dispatch via middleware, or dedicated libraries like React Query that handle loading/error/data states, cancellation, retries, and caching. Always handle loading and error states alongside the success path.

Async state with loading/error and unmount guard

```
import { useState, useEffect } from 'react';

function Data({ id }) {
  const [state, setState] = useState({
    data: null, loading: true, error: null
  });

  useEffect(() => {
    let active = true;
    setState(s => ({ ...s, loading: true, error: null }));
    fetch(`/api/items/${id}`)
      .then(r => r.json())
      .then(data => {
        if (active) setState({ data, loading: false, error: null });
      })
      .catch(error => {
        if (active) setState(s => ({ ...s, loading: false, error }));
      });
    return () => { active = false; };
  }, [id]);

  if (state.loading) return <p>Loading...</p>;
  if (state.error) return <p>Error: {state.error.message}</p>;
  return <pre>{JSON.stringify(state.data)}</pre>;
}
```

Rendering & Performance

Q31. Can you explain what the Virtual DOM is and how React uses it?

The Virtual DOM (VDOM) is a lightweight, in-memory JavaScript representation of the real DOM. React keeps a copy of the UI as a tree of plain objects. When state changes, React builds a new VDOM tree and compares it with the previous one (diffing); the diff produces the minimal set of DOM mutations. Direct DOM manipulation is slow, so touching it as little as possible is the performance win. The VDOM lets you write declarative code while React optimises the actual updates.

JSX becomes a VDOM object tree

```
// A React element is a plain object (the VDOM node)
const vdom = (
  <div className="app">
    <h1>Hello</h1>
    <p>World</p>
  </div>
);

// Compiles to:
// {
// type: 'div',
// props: {
// className: 'app',
// children: [
// { type: 'h1', props: { children: 'Hello' } },
// { type: 'p', props: { children: 'World' } }
// ]
// }
// }

// React diffs old vs new VDOM and updates only
// the changed DOM nodes
```

Q32. What do you understand by reconciliation in React? Why is it important?

Reconciliation is the process React uses to update the DOM to match the latest VDOM. When state or props change, React creates a new element tree and compares it with the previous one using a diffing algorithm. The algorithm assumes elements of different types produce different trees, and uses keys to match list items across renders. Reconciliation is important because it computes the minimal set of changes, keeping the real DOM in sync efficiently instead of re-rendering everything.

Type, props, and keys drive reconciliation

```
// Same type, different props -> React updates
// only the changed attributes
<div className="box" /> // before
<div className="box active" /> // after: only className changes

// Different types -> React tears down old, builds new
<span>Text</span> // before
<p>Text</p> // after: <span> removed, <p> created

// Keys help reconciliation match list items
{items.map(item => (
  <Item key={item.id} data={item} />
))}
```

Q33. How does React.memo help improve performance?

React.memo is a higher-order component that memoises a functional component. It does a shallow comparison of props; if they haven't changed, it skips re-rendering and reuses the last rendered output. This helps when a parent re-renders often but a child's props rarely change. For deep prop comparisons you can pass a custom comparison function, but shallow is usually best. Pair React.memo with useCallback/useMemo so passed props keep stable references.

React.memo with stable props via useCallback

```
import { memo, useState, useCallback } from 'react';

const ExpensiveList = memo(function List({ items, onSelect }) {
  console.log('List rendered');
  return (
    <ul>
      {items.map(i => (
        <li key={i.id} onClick={() => onSelect(i.id)}>
          {i.name}
        </li>
      ))}
    </ul>
  );
});

function App() {
  const [count, setCount] = useState(0);
  const [items] = useState([{ id: 1, name: 'A' }]);
  // Without useCallback, onSelect is new each render
  // -> memo breaks
  const handleSelect = useCallback(id => {
    console.log(id);
  }, []);

  return (
    <>
      <button onClick={() => setCount(c => c + 1)}>
        {count}
      </button>
      <ExpensiveList items={items} onSelect={handleSelect} />
    </>
```

```
  );
}
```

Q34. If I ask you to optimize a slow React application, what techniques would you use?

First profile with React DevTools Profiler to find wasted renders. Then apply: memoise expensive components (React.memo), keep callbacks stable (useCallback), memoise expensive computations (useMemo), split code with React.lazy + Suspense, virtualise long lists (react-window), avoid inline object/function props, move state down so updates are local, use keys correctly, and consider a selector-based store (Redux/Zustand). Also debouncing/throttling rapid events and deferring non-critical work with useTransition.

Multiple optimisation techniques together

```
import {
  memo, useMemo, useCallback,
  useState, useTransition, Suspense, lazy
} from 'react';

const HeavyChart = lazy(() => import('./HeavyChart'));

const Row = memo(function Row({ item, onClick }) {
  return <li onClick={() => onClick(item.id)}>{item.name}</li>;
});

function App({ items }) {
  const [query, setQuery] = useState('');
  const [isPending, startTransition] = useTransition();

  const filtered = useMemo(
    () => items.filter(i => i.name.includes(query)),
    [items, query]
  );
  const onClick = useCallback(id => console.log(id), []);

  return (
    <Suspense fallback={<p>Loading chart...</p>}>
      <input value={query}
        onChange={e => startTransition(() =>
          setQuery(e.target.value)
        )} />
      {isPending && <p>Updating...</p>}
      <ul>
        {filtered.map(i => (
          <Row key={i.id} item={i} onClick={onClick} />
        ))}
      </ul>
      <HeavyChart data={filtered} />
    </Suspense>
  );
}
```

Q35. What do you mean by code splitting and lazy loading in React?

Code splitting breaks your bundle into smaller chunks loaded on demand, reducing the initial download. React.lazy lets you render a dynamic import as a regular component, loading it only when first rendered. Suspense provides a fallback (like a spinner) while the chunk loads. This is commonly applied to route-level components so the user downloads each page's code only when they navigate to it.

Route-level code splitting with lazy + Suspense

```
import { lazy, Suspense } from 'react';
import {
  BrowserRouter, Routes, Route, Link
```

```
} from 'react-router-dom';

// Each route loads its code on demand
const Home = lazy(() => import('./Home'));
const Dashboard = lazy(() => import('./Dashboard'));

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/dash">Dashboard</Link>
      </nav>
      <Suspense fallback={<div>Loading page...</div>}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/dash" element={<Dashboard />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}
```

Q36. How would you optimize a slow React application?

Start by measuring with the Profiler to avoid premature optimisation. Common wins: virtualise large lists, memoise children and expensive computations, prevent unnecessary re-renders by lifting state correctly or splitting context, use stable prop references, code-split routes, and move heavy synchronous work off the main thread (Web Workers) or defer it with useTransition/useDeferredValue. Also ensure production builds and check for accidental re-renders from new object/array literals in props.

useDeferredValue keeps input responsive

```
import { useDeferredValue, useMemo, useState } from 'react';

function SearchResults({ allItems }) {
  const [query, setQuery] = useState('');
  // Defer the expensive filter so typing stays responsive
  const deferredQuery = useDeferredValue(query);
  const results = useMemo(
    () => allItems.filter(i => i.includes(deferredQuery)),
    [allItems, deferredQuery]
  );
  return (
    <>
      <input value={query}
        onChange={e => setQuery(e.target.value)} />
      <ul>
        {results.map((r, i) => <li key={i}>{r}</li>)}
      </ul>
    </>
  );
}
```

Q37. What is the difference between React.PureComponent and React.Component?

Both are base classes for class components, but PureComponent implements shouldComponentUpdate with a shallow comparison of props and state, skipping re-renders when nothing changed. React.Component always re-renders when parent re-renders (unless you write your own shouldComponentUpdate). PureComponent is a built-in optimisation for simple, shallow-equal data; for function components the equivalent is React.memo.

Component vs PureComponent vs React.memo

```
// React.Component: always re-renders
class Regular extends React.Component {
  render() {
    return <div>{this.props.name}</div>;
  }
}

// React.PureComponent: skips re-render if
// props/state are shallowly equal
class Optimised extends React.PureComponent {
  render() {
    return <div>{this.props.name}</div>;
  }
}

// Function component equivalent
const OptimisedFn = React.memo(
  function OptimisedFn({ name }) {
    return <div>{name}</div>;
  }
);
```

Q38. How does React handle re-rendering when state or props change?

When a component's state changes via setState/useReducer, or when its parent passes new props, React marks the component as dirty and schedules a re-render. During reconciliation it builds a new VDOM subtree and diffs it against the previous one, applying the minimal DOM changes. By default, a re-render also re-renders all child components; you can skip children with React.memo, shouldComponentUpdate, or useMemo/useCallback to keep prop references stable.

Unstable vs stable props and re-renders

```
function Parent() {
  const [count, setCount] = useState(0);
  // count changes -> Parent re-renders
  // -> Child re-renders (new prop literal)
  return (
    <>
      <button onClick={() => setCount(c => c + 1)}>
        {count}
      </button>
      <Child data={{ id: 1 }} /> {/* new object each render */}
    </>
  );
}

// Fix: memoise the prop so Child can skip
const Child = React.memo(function Child({ data }) {
  return <p>{data.id}</p>;
});

function FixedParent() {
  const [count, setCount] = useState(0);
  const data = useMemo(() => ({ id: 1 }), []); // stable
  return (
    <>
      <button onClick={() => setCount(c => c + 1)}>
        {count}
      </button>
      <Child data={data} />
    </>
  );
}
```

Q39. What is the difference between controlled and uncontrolled components in terms of rendering?

A controlled component re-renders on every keystroke because each change updates state, which triggers a re-render. This gives live access to the value (for validation, formatting) at the cost of a render per keystroke. An uncontrolled component does not re-render on each keystroke because the value lives in the DOM via a ref; React only reads it when needed (e.g. on submit). For large forms, uncontrolled inputs or debounced controlled inputs reduce render churn.

Controlled re-renders each keystroke; uncontrolled does not

```
import { useState, useRef } from 'react';

// Controlled: re-renders on every keystroke
function Controlled() {
  const [text, setText] = useState('');
  return (
    <input
      value={text}
      onChange={e => setText(e.target.value)}
    />
  );
}

// Uncontrolled: no re-render while typing
function Uncontrolled() {
  const ref = useRef(null);
  return (
    <form onSubmit={e => {
      e.preventDefault();
      console.log(ref.current.value);
    }}>
      <input ref={ref} defaultValue="" />
    </form>
  );
}
```

Q40. How do useMemo and useCallback help improve performance in React?

useMemo caches the result of a calculation so it isn't recomputed unless dependencies change; this prevents expensive work on every render and keeps derived values referentially stable. useCallback caches a function reference so the same instance is reused across renders, which prevents child components wrapped in React.memo from re-rendering when the parent re-renders. Both avoid unnecessary work but should be applied where profiling shows a real benefit, not on every value.

useMemo + useCallback keep memoised children efficient

```
import { useState, useMemo, useCallback, memo } from 'react';

const Child = memo(function Child({ total, onAdd }) {
  console.log('Child rendered');
  return <button onClick={onAdd}>Total: {total}</button>;
});

function Parent({ items }) {
  const [extra, setExtra] = useState(0);

  // useMemo: cache expensive sum; stable reference
  const total = useMemo(
    () => items.reduce((s, i) => s + i.price, 0) + extra,
    [items, extra]
  );

  // useCallback: stable handler so memoised
  // Child skips re-render
```

```
  const onAdd = useCallback(
    () => setExtra(e => e + 1),
    []
  );

  return <Child total={total} onAdd={onAdd} />;
}
```

Arrays & Lists

Q41. How do you render a list of items in React?

Use the JavaScript .map() method on the array, returning a JSX element for each item. Each element needs a unique key prop so React can track items across re-renders. Avoid .forEach() (it returns undefined and renders nothing). You can map over arrays stored in state or props.

Rendering a list with map() and keys

```
function FruitList({ fruits }) {
  return (
    <ul>
      {fruits.map(fruit => (
        <li key={fruit.id}>{fruit.name}</li>
      ))}
    </ul>
  );
}

// Usage
<FruitList fruits={[
  { id: 1, name: 'Mango' },
  { id: 2, name: 'Guava' }
]} />
```

Q42. Why is the key prop important in React lists?

Keys give each list item a stable identity so React can match elements between renders during reconciliation. Without a stable key, React may reuse the wrong DOM nodes, causing subtle bugs (e.g. input state sticking to the wrong item) and inefficient updates. Use a unique, stable value from your data (like an id). Avoid array indices when the list can be reordered, inserted, or deleted, because that shifts which item each index points to.

Why stable keys matter

```
// Good: stable unique id as key
{users.map(u => (
  <li key={u.id}>{u.name}</li>
))}

// Problematic: index as key (order can change)
{users.map((u, i) => (
  <li key={i}>{u.name}</li>
))}

// What goes wrong with index keys:
// Delete the first user -> React thinks user at
// index 0 is the same component, so input fields or
// animation state attached to it stay on the wrong item.
```

Q43. What happens if we use the array index as a key in React?

Using index as key works only for static lists that never reorder, insert, or delete. When items are added or removed, indices shift, so React matches components to the wrong data. This causes UI bugs like input values staying with the wrong row, broken animations, and unnecessary DOM updates. React may also warn in development. Always prefer a stable unique id from the data.

Index keys cause state leaks; id keys fix it

```
// Buggy with index keys when deleting
function TodoList({ todos }) {
  return todos.map((todo, i) => (
    <div key={i}>
      {/* state leaks to wrong row after delete */}
      <input defaultValue={todo.text} />
      <button onClick={() => remove(todo.id)}>
        Delete
      </button>
    </div>
  ));
}

// Fixed: stable id
function TodoList({ todos }) {
  return todos.map(todo => (
    <div key={todo.id}>
      <input defaultValue={todo.text} />
      <button onClick={() => remove(todo.id)}>
        Delete
      </button>
    </div>
  ));
}
```

Q44. How do you conditionally render list items in React?

Filter the array first with .filter(), then .map() the result, or conditionally return null inside the map. Filtering before mapping is cleaner and avoids rendering wrappers. You can combine filter and map in a chain, and the result still needs keys on the rendered elements.

Filter then map, or conditional return inside map

```
import { useState, useMemo } from 'react';

function ProductList({ products }) {
  const [inStockOnly, setInStockOnly] = useState(true);

  const visible = useMemo(() => {
    let list = products;
    if (inStockOnly) list = list.filter(p => p.inStock);
    return list;
  }, [products, inStockOnly]);

  return (
    <>
      <button onClick={() => setInStockOnly(s => !s)}>
        {inStockOnly ? 'Show all' : 'Show in-stock only'}
      </button>
      <ul>
        {visible.map(p => <li key={p.id}>{p.name}</li>)}
      </ul>
    </>
  );
}

// Alternative: conditional return inside map
```

```
{users.map(u =>
  u.active ? <li key={u.id}>{u.name}</li> : null
)}
```

Q45. How do you update or remove an item from a list in React state?

Never mutate the array directly (e.g. push/splice) because React won't detect the change. Instead, create a new array: use .map() to update an item, and .filter() to remove one. Always pass a new array reference so React re-renders. Use the functional form of setState to safely base the update on the previous state.

Update with map, remove with filter, add with spread

```
function TodoApp() {
  const [todos, setTodos] = useState([
    { id: 1, text: 'Learn React', done: false },
    { id: 2, text: 'Build app', done: false },
  ]);

  // Update: replace matching item, keep others
  const toggle = (id) => setTodos(prev =>
    prev.map(t =>
      t.id === id ? { ...t, done: !t.done } : t
    )
  );

  // Remove: filter out the matching item
  const remove = (id) => setTodos(prev =>
    prev.filter(t => t.id !== id)
  );

  // Add: spread existing then append
  const add = (text) => setTodos(prev =>
    [...prev, { id: Date.now(), text, done: false }]
  );

  return todos.map(t => (
    <div key={t.id}>
      <span style={{
        textDecoration: t.done ? 'line-through' : ''
      }}>
        {t.text}
      </span>
      <button onClick={() => toggle(t.id)}>Toggle</button>
      <button onClick={() => remove(t.id)}>Delete</button>
    </div>
  ));
}
```

Q46. What are some performance tips when rendering large lists in React?

Use windowing/virtualisation so only visible rows are in the DOM (react-window, react-virtual). Memoise row components with React.memo and pass stable callbacks. Avoid inline functions and objects as props. Keep keys stable and unique. Paginate or lazy-load data. For filtering/sorting, memoise with useMemo and debounce the search input so you don't recompute on every keystroke.

Virtualised, memoised large list

```
import { useMemo, useState, memo, useCallback } from 'react';
import { FixedSizeList as List } from 'react-window';

const Row = memo(function Row({ index, style, data }) {
  return <div style={style}>{data[index].name}</div>;
});
```

```
function BigList({ items }) {
  const [query, setQuery] = useState('');
  const filtered = useMemo(
    () => items.filter(i => i.name.includes(query)),
    [items, query]
  );
  const rowRenderer = useCallback(({ index, style }) => (
    <Row index={index} style={style} data={filtered} />
  ), [filtered]);

  return (
    <>
      <input value={query}
        onChange={e => setQuery(e.target.value)} />
      <List height={400} itemCount={filtered.length}
        itemSize={35} width={300}>
        {rowRenderer}
      </List>
    </>
  );
}
```

Q47. How would you render a nested list in React?

Render the outer array with map, and inside each outer item render its inner array with another map. Give each level a unique key. You can extract the inner rendering into its own component for clarity. Keys only need to be unique among siblings, not globally.

Nested list with two levels of map

```
function Departments({ departments }) {
  return (
    <ul>
      {departments.map(dept => (
        <li key={dept.id}>
          {dept.name}
          <ul>
            {dept.employees.map(emp => (
              <li key={emp.id}>{emp.name}</li>
            ))}
          </ul>
        </li>
      ))}
    </ul>
  );
}

const data = [
  {
    id: 'd1', name: 'Engineering',
    employees: [
      { id: 'e1', name: 'Aarav' },
      { id: 'e2', name: 'Priya' }
    ]
  },
  {
    id: 'd2', name: 'Sales',
    employees: [{ id: 'e3', name: 'Ravi' }]
  },
];
<Departments departments={data} />
```

Q48. How do you handle dynamic addition of items in a list?

Store the list in state and add new items by creating a new array with the existing items plus the new one (using spread or concat). Generate a unique id for the new item (Date.now(), uuid, or an incrementing counter). Controlled inputs feed the new item's data. Never mutate the existing array directly.

Adding items with a unique id via spread

```
import { useState } from 'react';

function TaskManager() {
  const [tasks, setTasks] = useState([]);
  const [input, setInput] = useState('');

  const addTask = () => {
    const text = input.trim();
    if (!text) return;
    setTasks(prev => [
      ...prev,
      { id: crypto.randomUUID(), text }
    ]);
    setInput('');
  };

  return (
    <>
      <input
        value={input}
        onChange={e => setInput(e.target.value)}
        onKeyDown={e => e.key === 'Enter' && addTask()}
      />
      <button onClick={addTask}>Add</button>
      <ul>
        {tasks.map(t => <li key={t.id}>{t.text}</li>)}
      </ul>
    </>
  );
}
```

Q49. How do you handle dynamic sorting or filtering of lists in React?

Keep the raw data and the current sort/filter settings in state. Derive the displayed list with useMemo so you don't recompute on every render. For sorting, use a copy of the array (sort mutates in place) and a comparator that reads the selected sort field and direction. For filtering, chain .filter() before sorting. Debounce text filters to avoid recomputing on each keystroke.

Filter and sort derived with useMemo

```
import { useState, useMemo } from 'react';

function SortableProducts({ products }) {
  const [sortKey, setSortKey] = useState('name');
  const [asc, setAsc] = useState(true);
  const [query, setQuery] = useState('');

  const displayed = useMemo(() => {
    let list = products.filter(p =>
      p.name.toLowerCase().includes(query.toLowerCase())
    );
    list = [...list].sort((a, b) => {
      const cmp = a[sortKey] > b[sortKey] ? 1
                : a[sortKey] < b[sortKey] ? -1 : 0;
      return asc ? cmp : -cmp;
    });
    return list;
  }, [products, sortKey, asc, query]);
```

```
  return (
    <>
      <input value={query}
        onChange={e => setQuery(e.target.value)} />
      <select value={sortKey}
        onChange={e => setSortKey(e.target.value)}>
        <option value="name">Name</option>
        <option value="price">Price</option>
      </select>
      <button onClick={() => setAsc(a => !a)}>
        {asc ? 'Asc' : 'Desc'}
      </button>
      <ul>
        {displayed.map(p => (
          <li key={p.id}>{p.name} - Rs.{p.price}</li>
        ))}
      </ul>
    </>
  );
}
```

Q50. What is the difference between rendering lists with map() vs forEach()?

map() returns a new array of elements, which React can render when placed inside JSX. forEach() returns undefined, so it renders nothing; it is for side effects only. Always use map() (or another array method that returns an array, like flatMap) when rendering lists. You can also use for loops to build an array first, then render it, but map() is the idiomatic React approach.

map() renders; forEach() does not

```
// Correct: map returns an array of elements
function Good({ items }) {
  return (
    <ul>
      {items.map(i => <li key={i.id}>{i.name}</li>)}
    </ul>
  );
}

// Wrong: forEach returns undefined -> renders nothing
function Bad({ items }) {
  return (
    <ul>
      {items.forEach(i => <li key={i.id}>{i.name}</li>)}
    </ul>
  ); // empty list!
}

// Alternative: build array with a loop, then render
function Alt({ items }) {
  const list = [];
  for (const i of items) {
    list.push(<li key={i.id}>{i.name}</li>);
  }
  return <ul>{list}</ul>;
}
```

Forms & Events

Q51. How are forms handled in React compared to plain HTML?

In plain HTML, the form's data lives in the DOM and is submitted via a full page reload. In React, you typically control inputs with state (controlled components), so React is the source of truth and you handle submission in JavaScript, preventing the default reload. This enables live validation, dynamic enabling/disabling of the submit button, and immediate feedback. Uncontrolled inputs (using refs) are also possible when you want the DOM to manage the value.

A controlled React form with state

```
import { useState } from 'react';

function ContactForm() {
  const [form, setForm] = useState({
    name: '', email: '', message: ''
  });

  const handleChange = (e) =>
    setForm(prev => ({
      ...prev,
      [e.target.name]: e.target.value
    }));

  const handleSubmit = (e) => {
    e.preventDefault(); // stop page reload
    console.log('Submitting:', form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name}
        onChange={handleChange} placeholder="Name" />
      <input name="email" value={form.email}
        onChange={handleChange} placeholder="Email" />
      <textarea name="message" value={form.message}
        onChange={handleChange} />
      <button type="submit">Send</button>
    </form>
  );
}
```

Q52. How do you prevent the default form submission behavior in React?

Call e.preventDefault() in the onSubmit handler. By default, submitting an HTML form triggers a page reload and a GET request to the action URL. In React you almost always want to handle the data in JavaScript instead, so you prevent the default, then read state, validate, and send the data to your API. This is also how you enable custom validation and loading states.

e.preventDefault() stops the reload

```
function LoginForm() {
  const [email, setEmail] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault(); // prevents browser reload/GET
    if (!email.includes('@')) {
      alert('Enter a valid email');
      return;
    }
    login(email); // proceed with API call
  };
```

```
  return (
    <form onSubmit={handleSubmit}>
      <input value={email}
        onChange={e => setEmail(e.target.value)}
        type="email" />
      <button type="submit">Log in</button>
    </form>
  );
}
```

Q53. What is event bubbling and how can you stop it in React?

Event bubbling means an event fired on a child propagates up to its ancestors. In React (which uses synthetic events), a click on a button inside a div will trigger both the button's onClick and the div's onClick. To stop this, call e.stopPropagation() in the child's handler. This is useful when a modal's inner content should not close the modal's backdrop handler.

stopPropagation prevents the backdrop handler

```
function Modal() {
  const close = () => console.log('closing modal');

  return (
    {/* click backdrop closes */}
    <div className="backdrop" onClick={close}>
      {/* stopPropagation prevents close */}
      <div className="content"
        onClick={e => e.stopPropagation()}>
        <h2>Modal</h2>
        <button onClick={() => console.log('clicked')}>
          Action
        </button>
      </div>
    </div>
  );
}
```

Q54. How do you handle multiple input fields in a single form in React?

Give each input a name attribute and store all field values in a single state object. In one shared onChange handler, update only the changed field using computed property names: [e.target.name]: e.target.value. This avoids writing a separate handler for every field. For checkboxes you read e.target.checked, and for selects you read e.target.value.

One handler for text, select, and checkbox

```
import { useState } from 'react';

function RegistrationForm() {
  const [form, setForm] = useState({
    name: '', email: '', role: 'user', subscribe: false
  });

  // one handler for every field
  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setForm(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value
    }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
```

```
    console.log(form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name}
        onChange={handleChange} />
      <input name="email" value={form.email}
        onChange={handleChange} />
      <select name="role" value={form.role}
        onChange={handleChange}>
        <option value="user">User</option>
        <option value="admin">Admin</option>
      </select>
      <input type="checkbox" name="subscribe"
        checked={form.subscribe} onChange={handleChange} />
      <button type="submit">Register</button>
    </form>
  );
}
```

Q55. How do you reset form fields after submission in React?

For a controlled form, reset the state object back to its initial values after submission. You can also call e.target.reset() on the form element (which resets controlled inputs only if they use defaultValue). For uncontrolled forms, e.target.reset() clears all DOM inputs. Storing the initial state as a constant keeps the reset clean and consistent.

Resetting a controlled form to initial values

```
import { useState } from 'react';

const INITIAL = { name: '', email: '' };

function SignupForm() {
  const [form, setForm] = useState(INITIAL);

  const handleChange = (e) =>
    setForm(prev => ({
      ...prev,
      [e.target.name]: e.target.value
    }));

  const handleSubmit = (e) => {
    e.preventDefault();
    submitToApi(form);
    setForm(INITIAL); // reset controlled fields
    // or: e.target.reset(); // for uncontrolled inputs
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name}
        onChange={handleChange} />
      <input name="email" value={form.email}
        onChange={handleChange} />
      <button type="submit">Sign up</button>
    </form>
  );
}
```

React Router

Q56. What is React Router and why is it used?

React Router is the standard routing library for React. It lets you define routes that map URLs to components, enabling navigation between views without a full page reload (client-side routing). This keeps a single-page application fast and lets users use the browser's back/forward buttons, share URLs, and bookmark pages. It supports nested routes, dynamic parameters, redirects, and route guards.

Basic routing setup

```
import {
  BrowserRouter, Routes, Route
} from 'react-router-dom';
import Home from './Home';
import About from './About';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Q57. Difference between the anchor tag and Link in React Router?

A plain anchor tag triggers a full page reload: the browser requests the new URL from the server and re-renders the whole app, losing in-memory state. React Router's Link intercepts the click, updates the URL via the History API, and renders only the matching route, preserving state and avoiding a reload. Link also enables prefetching and active styling. Use Link for internal navigation; use a plain anchor only for external URLs.

Link for internal, anchor for external

```
import { Link } from 'react-router-dom';

function Nav() {
  return (
    <nav>
      {/* Internal: no reload, preserves state */}
      <Link to="/about">About</Link>
      <Link to="/contact" className="nav-link">
        Contact
      </Link>

      {/* External: must use plain anchor */}
      <a href="https://react.dev"
        target="_blank" rel="noreferrer">
        React docs
      </a>
    </nav>
  );
}
```

Q58. What are the main components of React Router?

The key building blocks are: BrowserRouter (or HashRouter) which keeps the UI in sync with the URL; Routes (the container that matches the current URL) and Route (defines a path and the element to render); Link and NavLink for navigation; useNavigate for programmatic navigation; useParams for reading URL parameters; and useLocation for the current location object. Outlet renders nested routes.

Core router components together

```
import {
  BrowserRouter, Routes, Route, Link, NavLink,
  Outlet, useNavigate, useParams
} from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <NavLink to="/users" activeClassName="active">
          Users
        </NavLink>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/users" element={<Users />}>
          <Route path=":id" element={<UserDetail />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}

function Users() {
  return (
    <>
      <h2>Users</h2>
      <Outlet /> {/* nested routes render here */}
    </>
  );
}

function UserDetail() {
  const { id } = useParams();
  const navigate = useNavigate();
  return (
    <button onClick={() => navigate('/')}>Home</button>
  );
}
```

Q59. What is the difference between useHistory and useNavigate?

useHistory was the hook in React Router v5; it returned a history object with methods like push, replace, and goBack. In React Router v6, history was replaced by the navigation API, and the equivalent hook is useNavigate, which returns a navigate function you call as navigate('/path') or navigate(-1) for back. v6's API is simpler and is the current standard.

v5 useHistory vs v6 useNavigate

```
// React Router v5 (older)
import { useHistory } from 'react-router-dom';
function OldLogin() {
  const history = useHistory();
  const login = () => history.push('/dashboard');
  const goBack = () => history.goBack();
  return <button onClick={login}>Login</button>;
}
```

```
// React Router v6 (current)
import { useNavigate } from 'react-router-dom';
function NewLogin() {
  const navigate = useNavigate();
  const login = () => navigate('/dashboard');
  const goBack = () => navigate(-1);
  const replace = () =>
    navigate('/login', { replace: true });
  return <button onClick={login}>Login</button>;
}
```

Q60. How do you implement nested routes in React Router?

In v6, define child Route elements inside a parent Route and render an Outlet in the parent component where the child should appear. The parent layout wraps the nested view. Nested paths are relative to the parent path. This lets you build layouts (e.g. a dashboard with a sidebar) where only the inner content changes.

Nested routes with Outlet

```
import {
  BrowserRouter, Routes, Route, Outlet, Link
} from 'react-router-dom';

function Dashboard() {
  return (
    <div className="dashboard">
      <nav>
        <Link to="stats">Stats</Link>
        <Link to="settings">Settings</Link>
      </nav>
      <Outlet /> {/* child route renders here */}
    </div>
  );
}

function Stats() { return <p>Statistics</p>; }
function Settings() { return <p>Settings</p>; }

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/dashboard" element={<Dashboard />}>
          <Route path="stats" element={<Stats />} />
          <Route path="settings" element={<Settings />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

Q61. What are route parameters and how do you access them?

Route parameters are dynamic segments of the URL, defined with a colon (e.g. /users/:id). They let a single route render different data based on the URL. You read them in the component with the useParams hook, which returns an object mapping each parameter name to its value.

Reading URL params with useParams

```
import { Routes, Route, useParams } from 'react-router-dom';

function App() {
  return (
```

```
    <Routes>
      <Route
        path="/products/:category/:id"
        element={<Product />}
      />
    </Routes>
  );
}

function Product() {
  const { category, id } = useParams();
  // URL: /products/electronics/42
  // -> { category: 'electronics', id: '42' }
  return (
    <p>Category: {category}, Product ID: {id}</p>
  );
}
```

Q62. How do you redirect a user in React Router?

In v6, use the Navigate component for declarative redirects (e.g. a guard route that sends unauthenticated users to /login), or useNavigate for programmatic redirects after an action. To replace instead of push history, set replace: true so the back button skips the redirected page. You can also redirect by rendering Navigate conditionally based on auth state.

Declarative Navigate vs programmatic useNavigate

```
import {
  Navigate, useNavigate, Routes, Route
} from 'react-router-dom';

// Declarative: guard a protected route
function ProtectedRoute({ user, children }) {
  if (!user) return <Navigate to="/login" replace />;
  return children;
}

function App() {
  return (
    <Routes>
      <Route path="/login" element={<Login />} />
      <Route path="/dashboard" element={
        <ProtectedRoute user={currentUser}>
          <Dashboard />
        </ProtectedRoute>
      } />
    </Routes>
  );
}

// Programmatic: after login success
function Login() {
  const navigate = useNavigate();
  const handleLogin = async () => {
    await auth();
    navigate('/dashboard', { replace: true });
  };
  return <button onClick={handleLogin}>Log in</button>;
}
```

Q63. What is the difference between client-side routing and server-side routing?

Server-side routing means every navigation sends a request to the server, which returns a fresh HTML page, causing a full reload and slower perceived transitions. Client-side routing (used by React Router) intercepts navigation in JavaScript, updates the URL via the History API, and swaps only the changed components, preserving state and enabling instant transitions. Server-side routing is better for SEO and initial load of content-heavy pages; client-side is better for app-like interactivity.

Server-side (anchor) vs client-side (Link)

```
// Server-side: each link reloads the page
<a href="/about">About</a>
// browser requests /about, server returns new HTML

// Client-side: no reload, React swaps the view
<Link to="/about">About</Link>
// JS updates URL, renders <About/> only

// Next.js combines both: file-based routing with
// SSR/SSG for SEO, plus client-side navigation
// between pages after hydration.
```

Q64. Difference between Switch and Routes in React Router.

Switch was the route-matching container in React Router v5; it rendered the first matching Route and ignored the rest. In v6 it was renamed to Routes, with a new element-based API (element prop instead of component/render), relative paths, and ranking-based matching that picks the best match rather than just the first. Routes is the v6 standard; Switch is deprecated.

v5 Switch vs v6 Routes

```
// React Router v5 (older) - Switch + component
import { Switch, Route } from 'react-router-dom';
<Switch>
  <Route exact path="/" component={Home} />
  <Route path="/about" component={About} />
</Switch>

// React Router v6 (current) - Routes + element
import { Routes, Route } from 'react-router-dom';
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
</Routes>
// Note: 'exact' is automatic in v6
```